

Universidad Rafael Landívar
Facultad de Ingeniería
Licenciatura en Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos Sección: 2
Docente: Ing. Luis Rojas



Semana 13

“API”

Estudiante: Sebastián Alexander
Villeda Reyna
Carné: 1032625

Guatemala 1 de octubre de 2026

A) Tipos de API

1. APIs públicas u Open APIs

Son interfaces que una organización ofrece a desarrolladores externos para que utilicen sus servicios. Generalmente cuentan con documentación que explica las operaciones disponibles, los datos necesarios y el formato de las respuestas.

Que una API sea pública no significa que sea gratuita o que permita acceso ilimitado. Puede exigir registro, una clave de identificación, límites de uso o el pago de una tarifa. Su principal ventaja es que facilita la creación de aplicaciones que aprovechen servicios de otras empresas.

Un ejemplo es Google Maps: sus APIs permiten incorporar mapas, buscar lugares y calcular rutas. Una aplicación de transporte ~~publica~~ puede utilizarlas para mostrar el recorrido entre el punto de partida y el destino. Fuente: Google Maps.

2. APIs Internas o privadas

Son interfaces diseñadas para utilizarse dentro de una organización. Conectan aplicaciones y servicios internos, y su acceso se restringe a usuarios o sistemas autorizados. Permiten compartir información sin exponer directamente todas las partes de una base de datos.

Un caso real es la API GraphQL empresarial de Netflix, que proporciona información a sus aplicaciones internas. Entre los conceptos que maneja se encuentran películas y actores. Esto ayuda a que diferentes herramientas de la empresa utilicen información relacionada de forma organizada.
Fuente: Netflix.

3. APIs de Socios o Partner APIs.

Son interfaces que se comparten con empresas, proveedores o colaboradores autorizados. Su acceso depende de una relación comercial, un proceso de registro o acuerdos que establecen que información puede consultarse y cómo debe utilizarse.

Un ejemplo es Amazon Selling Partner API. Permite a vendedores y aplicaciones autorizadas trabajar con información de pedidos, inventarios, pagos y envíos. Su uso está al permisos y requisitos de autorización. Fuente: Amazon.

4. APIs compuestas o Composite APIs

Son interfaces que agrupan varias operaciones dentro de una solicitud. Resultan útiles cuando una tarea necesita consultar o modificar varios recursos, especialmente si una operación depende del resultado de otra.

Un ejemplo real es Salesforce Composite API, que ejecuta varias solicitudes REST dentro de una petición. Además, permite utilizar el resultado de una operación en solicitudes posteriores. La gestión de errores depende de las opciones del servicio. Fuente: Sales Force.

b) Estilos Arquitectónicos y Mecanismos de Comunicación

1. REST

REST es un estilo arquitectónico que organiza la información en recursos, como usuarios como usuarios, productos o pedidos. Cada recurso se identifica mediante una dirección URL. En las APIs web se utilizan métodos HTTP para indicar la operación que se desea realizar.

GET permite consultar información, POST suele utilizarse para crear recursos, PUT para reemplazarlos, PATCH para modificarlos parcialmente y DELETE para eliminarlos. Las respuestas frecuentemente emplean JSON, aunque REST no obliga a utilizar ese formato.

Se utiliza explícitamente en aplicaciones web, móviles y tiendas virtuales. Por ejemplo, GET /Productos/5 permite solicitar los datos del producto número cinco.

2. GraphQL

GraphQL es un lenguaje de consulta para APIs y un sistema para ejecutar esas consultas. El cliente especifica los campos que necesita, y el servidor responde según esa selección. El servicio define mediante un esquema qué datos y operaciones están disponibles.

Se emplea cuando distintas pantallas o aplicaciones necesitan conjuntos diferentes de datos relacionados. Ayuda a evitar la transferencia de información innecesaria, aunque requiere controlar la complejidad de las consultas para no sobrecargar el servidor. Fuente: GraphQL.

3. SOAP

SOAP es un ~~protocolo~~ Protocolo para intercambiar mensajes estructurados en XML. Sus mensajes contienen un sobre que identifica el mensaje y un cuerpo con la información principal. También pueden incluir encabezados con datos adicionales para su procesamiento.

Se utilizan en integraciones empresariales, sistemas bancarios y aplicaciones que requieren estándares específicos de seguridad y mensajería. Por ejemplo, una aplicación envía solicitud XML para consultar una cuenta y recibir otra estructura XML con el resultado. Su estructura suele ser más extensa que la de mensajes sencillos en JSON.

4. GRPC

gRPC es un sistema de llamadas a procedimientos remotos. Permite que una aplicación invoque una función que se ejecuta en otro servidor, utilizando código cliente generado a partir de una definición del servicio.

Se usa especialmente en microservicios y sistemas que necesitan comunicación eficiente. Por ejemplo, un servicio de pedidos puede llamar a una función del servicio de inventario para verificar existencias. Su formato binario favorece intercambios compactos, aunque resultado resulta menos fácil de leer directamente que un mensaje de texto. Fuente: gRPC.

5. Web Socket

Web Socket es un protocolo que establece una conexión persistente un cliente y un servidor. Después de establecerla, ambos pueden enviar mensajes sin abrir una conexión por cada intercambio.

Esto permite que el servidor comunique cambios en cuanto ocurren, sin que el cliente tenga que consultar constantemente si existe información nueva.

Se utiliza en chats, videojuegos en línea, aplicaciones colaborativas y paneles que muestran datos en tiempo real.

Por ejemplo, en un chat, el servidor puede enviar inmediatamente un mensaje a los usuarios conectados. Para mantener el servicio funcionando correctamente, la aplicación debe manejar desconexiones y recuperar la conexión cuando sea necesario.

6. Webhooks

Los Webhooks son notificaciones automáticas que un sistema envía a otro cuando ocurre un evento. El sistema receptor proporciona una dirección web, y el emisor envía una solicitud HTTP, normalmente POST, con los datos del acontecimiento.

Se utilizan para informar sobre pagos, pedidos, cambios en archivos o actualizaciones en repositorios. Por ejemplo, cuando se confirman, la plataforma puede enviar un webhook a la tienda para que actualice el pedido.

C) Conceptos Relacionados

1. Payload

El Payload es el contenido útil que transporta un mensaje. En una API HTTP normalmente se refiere a los datos incluidos en el cuerpo de una solicitud o respuestas. Puede contener texto, objetos JSON, documentos XML o archivos.

Por ejemplo: al registrar un producto, el payload podría ser: `{ "nombre": "Cuaderno", "precio": 153 }`. Estos datos indican que producto se desea guardar. Los encabezados, como el que indica el tipo de contenido, describen cómo interpretar el mensaje.

2. Request

Request significa solicitud. Es el mensaje que un cliente envía a un servidor para consultar información o ejecutar una operación. Una solicitud HTTP incluye un método, una dirección y encabezados; también puede contener un cuerpo.

Por ejemplo, `GET /Producto/S` solicita los datos de un producto. En cambio, `POST /Producto` puede enviar los datos necesarios para crear uno nuevo. El servidor revisa la solicitud, valida los permisos y procesa la operación correspondiente.

3. Response

Response significa respuesta. Es el mensaje que el servidor devuelve después de recibir una solicitud. Contiene un código de estado, encabezados y, cuando

Corresponde 1 un cuerpo con información.

Por ejemplo, una consulta correcta puede recibir el estado 200 OK junto con los datos del producto, si se crea un recurso, puede recibirse 201 Created. Si el recurso solicitado no existe, puede devolverse 404 Not Found. Los códigos ayudan al cliente a interpretar el resultado.

4. cURL

cURL es una herramienta de línea de comandos que permiten transferir datos y realizar solicitudes a servidores. Se utiliza para probar APIs, enviar información, agregar encabezados y revisar respuestas.

Por ejemplo, `curl https://example.com` realiza una solicitud GET y muestra el contenido recibido. Para trabajar con APIs también pueden indicarse otros métodos, credenciales y datos en el cuerpo.

Es útil para comprobar el funcionamiento de un servicio y automatizar tareas mediante scripts. A diferencia de Postman, su uso principal se realiza mediante comandos escritos en una terminal. Fuente: cURL

D) Tipos de autorización en Postman

Postman permite configurar credenciales para probar APIs. La autenticación comprueba la identidad; la autorización determina los permisos.

1. No. Auth: envía solicitudes sin credenciales.
2. API key: Utiliza una clave en un encabezado o parámetro.
3. Basic Auth: Envía usuario y contraseña codificados en Base64, necesita HTTPS para protegerlos.
4. Bearer Token: incluyen un token en el encabezado Authorization.
5. JWT Bearer: genera y envía un token JWT firmado.
6. OAuth 1.0: firma solicitudes utilizando claves y tokens.
7. OAuth 2.0: permite obtener tokens con permisos de usuario.
8. Digest Auth: Responde a un desafío mediante cálculos criptográficos.
9. Hawk: Utiliza una firma basada en una clave compartida.

10. AWS: Signature: firma solicitudes para servicios de Amazon Web Services.

11. NTLM: Permite autenticarse en sistemas compatibles con este mecanismo de Windows.

12. Akamai Edge Grid: autentica solicitudes a APIs de Akamai.

13. ASAP: Utilizan tokens firmados para autenticar comunicaciones entre servicios.

La opción de heredar autorización permite reutilizar la configuración de una carpeta o colección. Por ejemplo, varias solicitudes pueden compartir un mismo OAuth Token. El método seleccionado debe coincidir con el que la API. Fuente: Postman.